CIS*3110 Operating Systems

Week 4

Lecture #7 Virtualization: Segmentation

Prof. Yulia Kotseruba

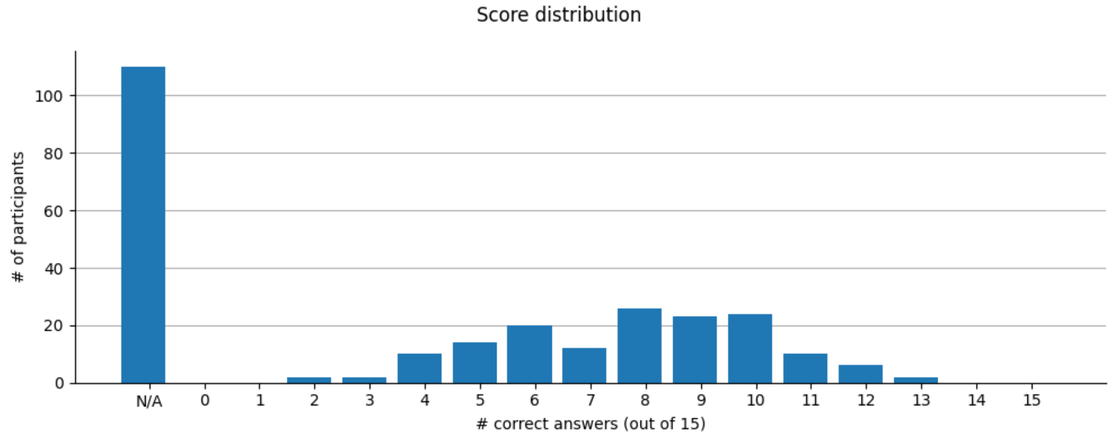
January 29, 2026

Quiz 5

- **61%** average (up from 57% in quiz 4)
- 3 hardest questions:
  - (21%) What is the output of this code? a) compiler error b) runtime error c) p=1 q=3 d) p=1 q=2
  - (28%) A=B=C=5s arrive 1s apart starting at t=0 in that order. What is C's response time under RR(q=5) a) 0 b) 5 c) 7 d) 10
  - (41%) Which of these schedulers is not fair-share? a) MLFQ b) stride c) RR d) EEVDF

```
int main() {
  int* p, q;
  int arr[3] = {1,2,3};
  p = arr;
  q = arr[1];
  printf("p=%d q=%d\n", p[0], q[1]);
}
```

error: subscripted value is not an array, pointer, or vector



int *a; = int* a;

int *p, *q; ≠ int* p, q;

both p and q are pointers to int

p is a pointer and q is just an int

Recap

- **Memory virtualization**
  - Processes see their memory as contiguous and virtually unlimited
  - Processes are not aware of their memory location in physical memory
  - OS manages allocation and access to memory to protect processes from each other and itself from processes
- **Address space** - a chunk of memory allocated by the OS for each process
  - processes can allocate/deallocate memory within that chunk
- **Memory API** - `malloc()`, `free()`

Recap

- `malloc()` - allocates
  - a **single contiguous unitialized** block of memory
  - on the **heap**
  - at **runtime**
- `free()` - deallocates
  - only memory **previously allocated by malloc**
  - any other pointer (not from malloc, not on the heap, invalid address) will result in something **very bad** (segfault, bus error, undefined behavior, bugs)
- to `free()` or not to `free()` all malloc'd memory?
  - better to always `free()` unless you know what you are doing

```
char *func() {
  char str[10];
  strcpy(str, "Hi!");
  return str;
}

int main(){
  char *s = func();
  s[2] = '?';
  printf("%s\n", s);

  int *i = malloc(sizeof(int));
  free(i);
  i[1] = 3;
  printf("%d\n", i[1]);
}
```

Is anything wrong with this code? Will it compile/run?

func() allocates a string on the stack and initializes it to "Hi!"

main() assigns the string returned by func() to variable s, modifies it to "Hi?" and prints it

allocates space for one int, frees it, then tries to write outside the bounds of that buffer

Freeing memory early

```
char *func() {          early.c
  char str[10];
  strcpy(str, "Hi!");
  return str;
}

int main(){
  char *s = func();
  s[2] = '?';
  printf("%s\n", s);

  int *i = malloc(sizeof(int));
  free(i);
  i[1] = 3;
  printf("%d\n", i[1]);
}
```

Is anything wrong with this code? Will it compile/run?

```
>>> ./early

-160600880
```

Will it compile and/or run?

code compiles (with warnings) and runs

pointer str is gone from the stack by the time it is printed in main

s is likely pointing to something else now

anything (or nothing) could be printed

after free(), pointer i still points to the same location, but the memory has been freed and could be overwritten by the time printf is called

Freeing freed memory

```
                    again.c
int main(){
  char *ptr =
  malloc(sizeof(char));
  *ptr = 'a';
  free(ptr);
  free(ptr);
}
```

Is anything wrong with this code? Will it compile/run?

```
>>> ./again
again(37221,0x20a77a240) malloc: *** error for object
0x6000001770030: pointer being freed was not allocated
again(37221,0x20a77a240) malloc: *** set a breakpoint in
malloc_error_break to debug
Abort trap: 6
```

free() expects a pointer to the valid address on the heap

but after the first free, the address is not valid anymore

SIGABRT (6) signal was sent when freeing a non-initialized/ corrupted/invalid pointer

calls abort() function - abnormal program termination, does not execute clean up

Freeing wrong memory

forbidden_free.c

```
int main() {
  int* p1;
  int m = 100;
  p1 = &m;

  free(p1);
  return 0;
}
```

Is anything wrong with this code? Will it compile/run?

```
>>> ./forbidden_free
forbidden_free(35920,0x20a77a240) malloc: *** error for
object 0x16cf02fcc: pointer being freed was not allocated
forbidden_free(35920,0x20a77a240) malloc: *** set a
breakpoint in malloc_error_break to debug
Abort trap: 6
```

p1 is pointing to something on the stack

malloc() expects a pointer to the valid address on the heap

SIGABRT (6) signal was sent when freeing a non-initialized/ corrupted/invalid pointer

Note: the exact problem (segfault, bus error, or abort signal) may be C implementation-dependent. You do not need to memorize which one will be triggered. Focus on recognizing invalid memory operations themselves.

`malloc()` + `free()`

- **Allocate enough memory** - avoid overwriting existing memory and buffer overflow
- **Initialize memory** - a random chunk of allocated memory can contain anything in it
- **Free memory** - avoid memory leaks*
- **Free only once** - on repeated free() the process will be aborted
- **Free only malloc'd memory** - the process attempting to free an invalid address will be aborted
- **Check malloc() return value**
  - `malloc()` returns NULL if unsuccessful. It is very unlikely in modern huge address spaces, but still good to check

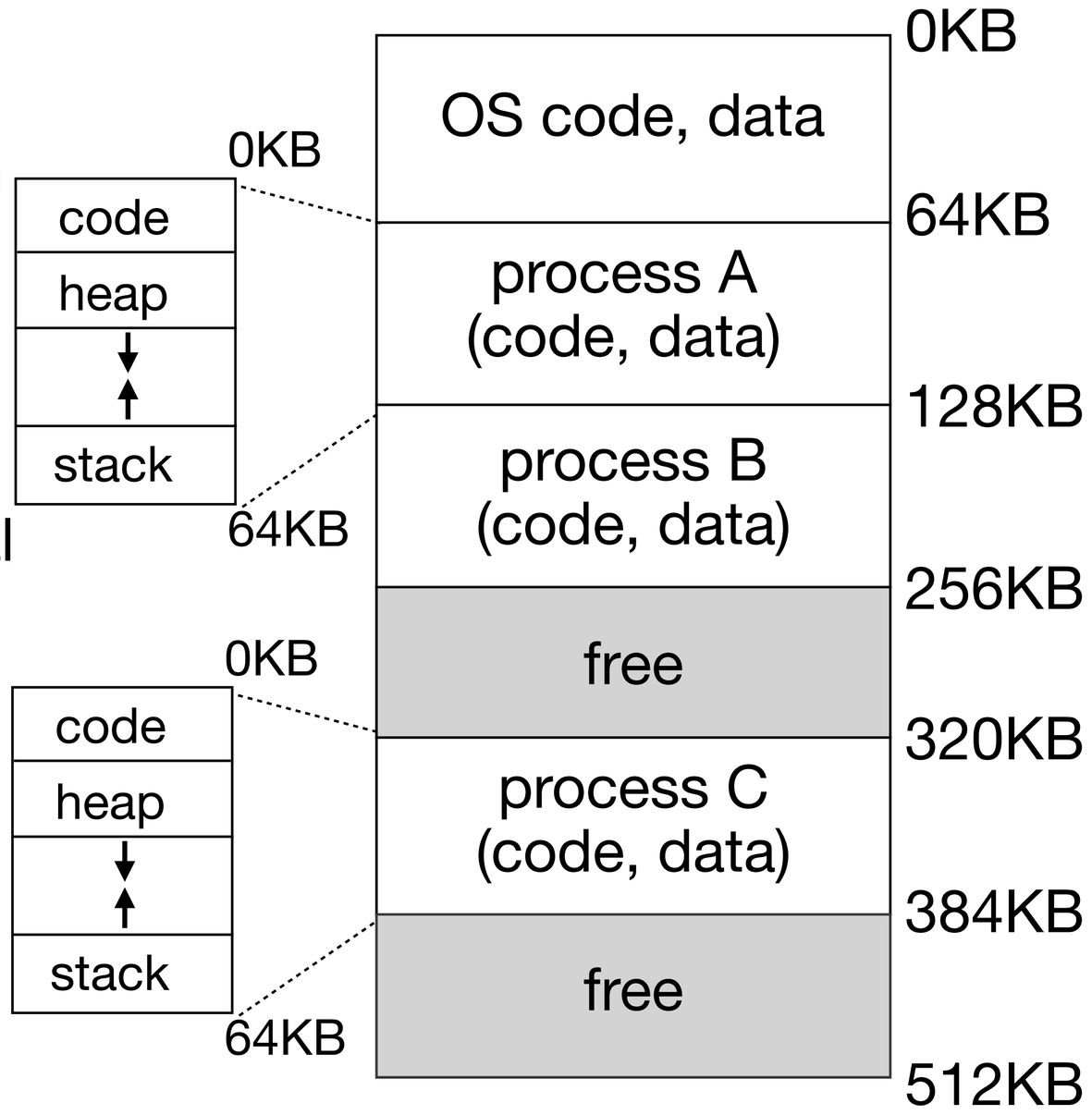
*unless you know what you are doing

`brk(),sbrk(),mmap()`

- `malloc()` and `free()` are NOT system calls, they are **library calls**
- malloc library is built on top of system calls
  - **`brk()`** - sets the location program's break (aka the address of the end of the heap)
  - **`sbrk()`** - increments the location set by brk()
  - **`mmap()`** - allocates larger memory blocks (>M_MMAP_THRESHOLD), slower than `brk()`
- **DO NOT** call `brk()`, `sbrk()` and `mmap()` directly
- `malloc()` and `free()` are simpler and standardized (can also be made thread-safe)

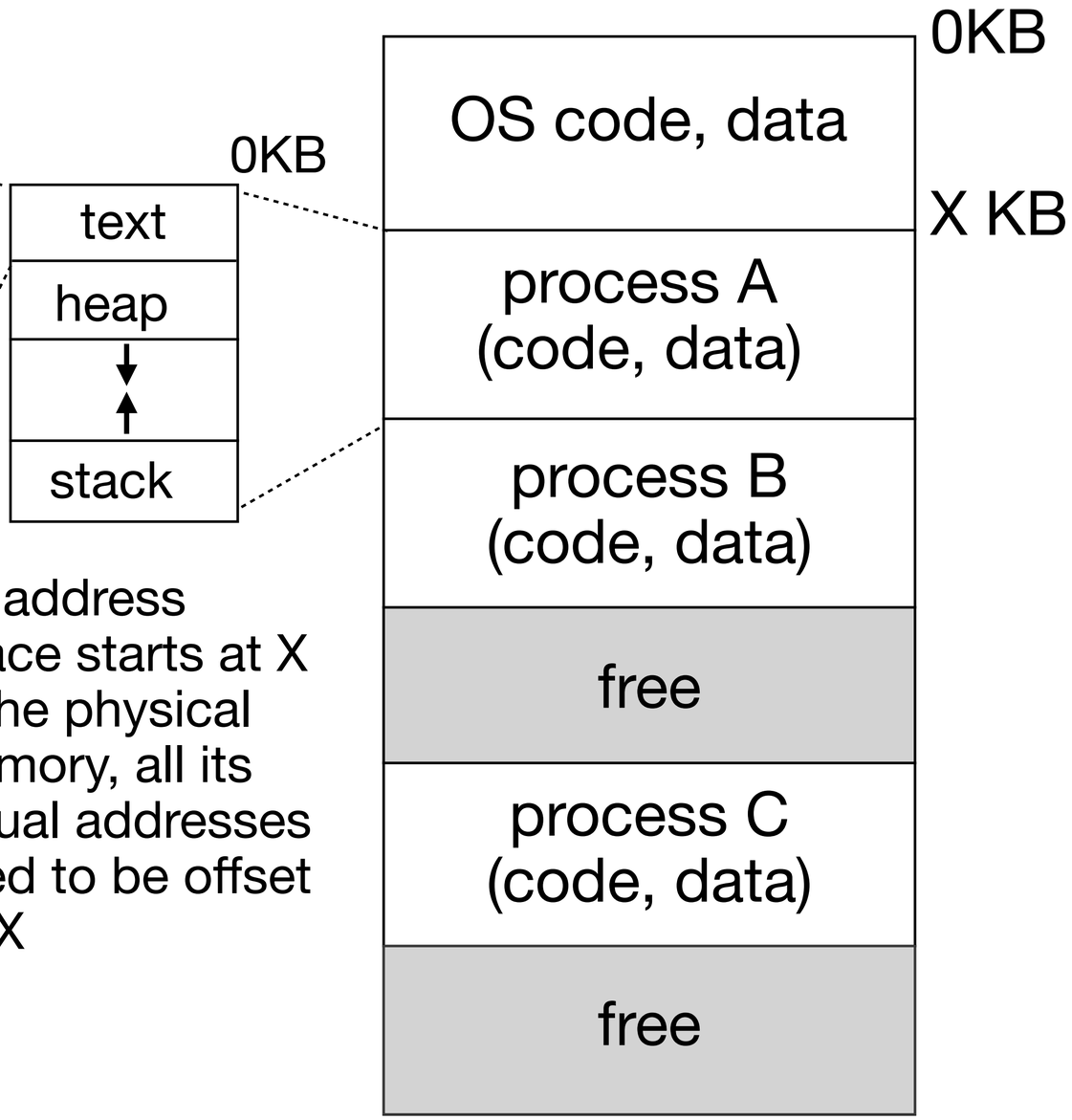
Address translation

- OS allocates a chunk of memory for each process in physical memory
- Processes see it as a virtual address space (e.g. starting at 0)
- Processes do not know where their virtual memory is located in the physical memory
- OS handles the translation



Address translation

```
0000000100000328      sub        sp, sp, #0x10
000000010000032c      mov        w8, #0xa
0000000100000330      str        w8, [sp, #0xc]
0000000100000334      ldr        w8, [sp, #0xc]
0000000100000338      add        w8, w8, #0x5
000000010000033c      str        w8, [sp, #0xc]
0000000100000340      mov        w0, #0x0
0000000100000344      add        sp, sp, #0x10
0000000100000348      ret
```



A's address space starts at X in the physical memory, all its virtual addresses need to be offset by X

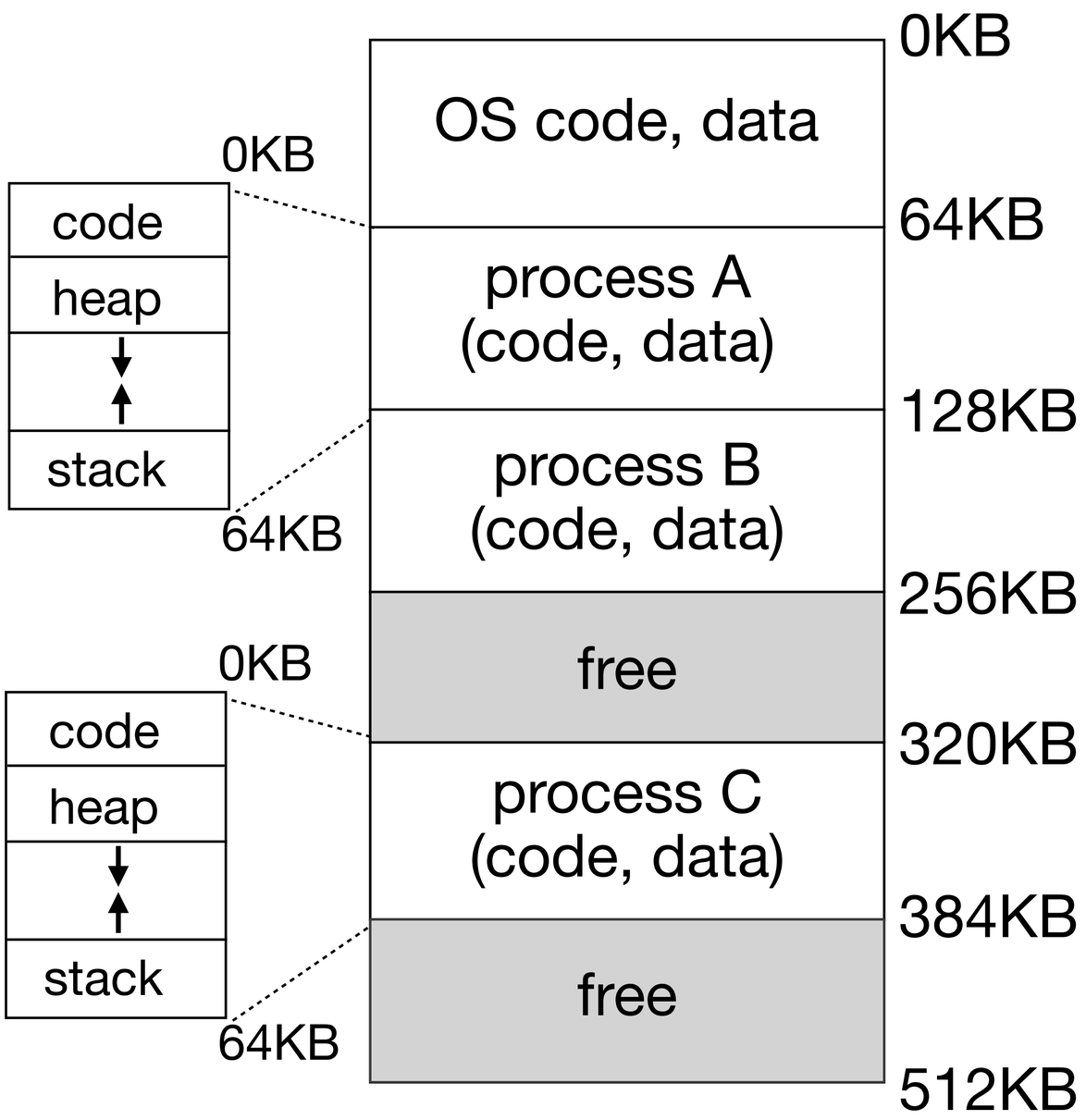
- When this program is run, it will get its own address space
- Instructions will be placed in the text section and executed one by one
- All memory references are w.r.t. to the virtual memory of the process
- How do we translate virtual address locations to physical locations in RAM?

Address translation

- **Problem #1**: How do we translate virtual addresses to physical addresses?
- **Option #0: static relocation**
  - at runtime, after memory has been allocated, a **loader** takes an executable and **rewrites all its addresses** with correct **offsets**
  - pros: purely software based - requires a loader
  - cons: no memory protection, difficult to relocate the binary again
    - it is hard to anticipate in advance whether the program will perform an illegal memory access at runtime (e.g. overwrite a portion of the OS itself or interfere with other processes)

Address translation

- **Problem #1**: How do we translate virtual addresses to physical addresses?
- **Option #2: dynamic relocation** (aka base and bounds)
  - **hardware-based** relocation
  - 2 registers per CPU
    - base - offset into physical memory
    - bounds - end of the address space in physical memory
  - physical address = virtual address + base
  - assert(physical address < bounds) - otherwise illegal memory access

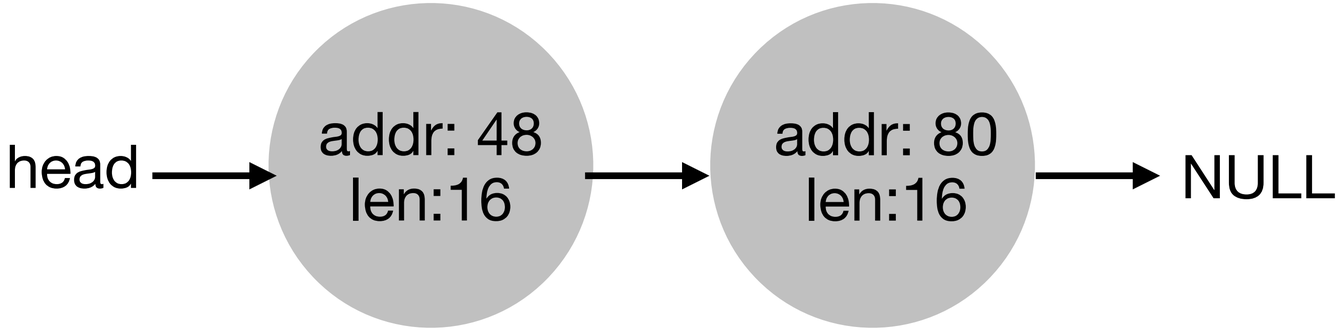


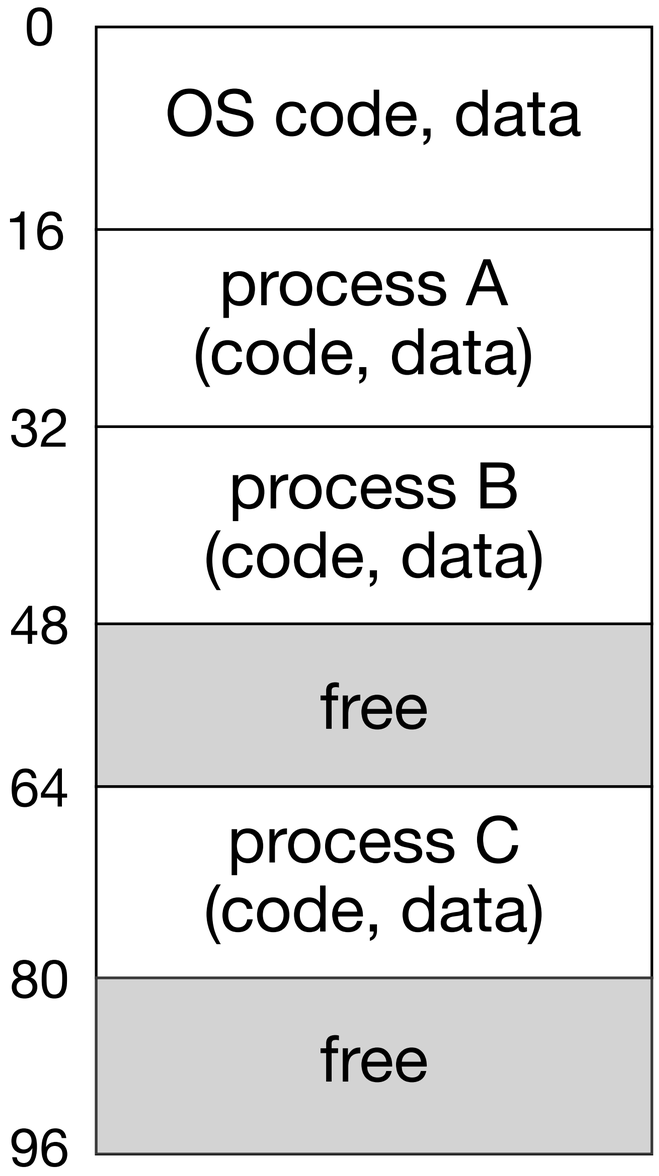
Base and bounds

- Requires **2 registers per CPU: base and bounds**
  - base is the offset and bounds is the limit
  - physical address = virtual address + base
  - assert(physical address < bounds)
- Is performed in **privileged** mode
  - OS is setting the values of base and bounds registers and puts them in PCB
  - MMU performs the address translation
  - OS generates exceptions when memory out of bounds is accessed

Address translation

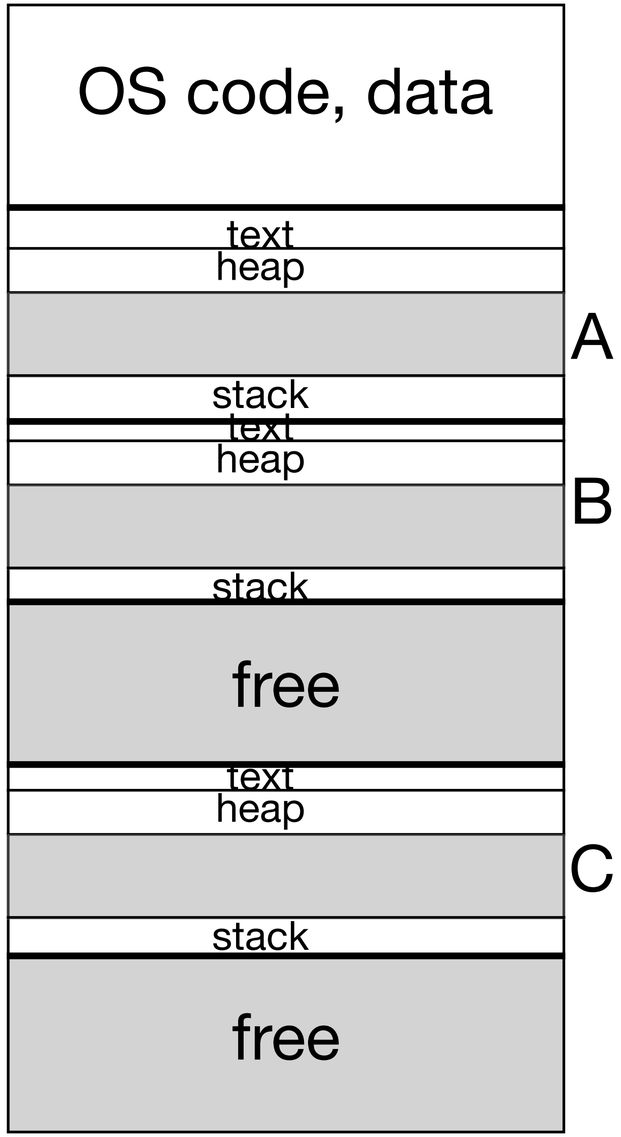
- **Problem #2**: What memory is available?
- **Solution: free list**
  - usually a linked list of free blocks





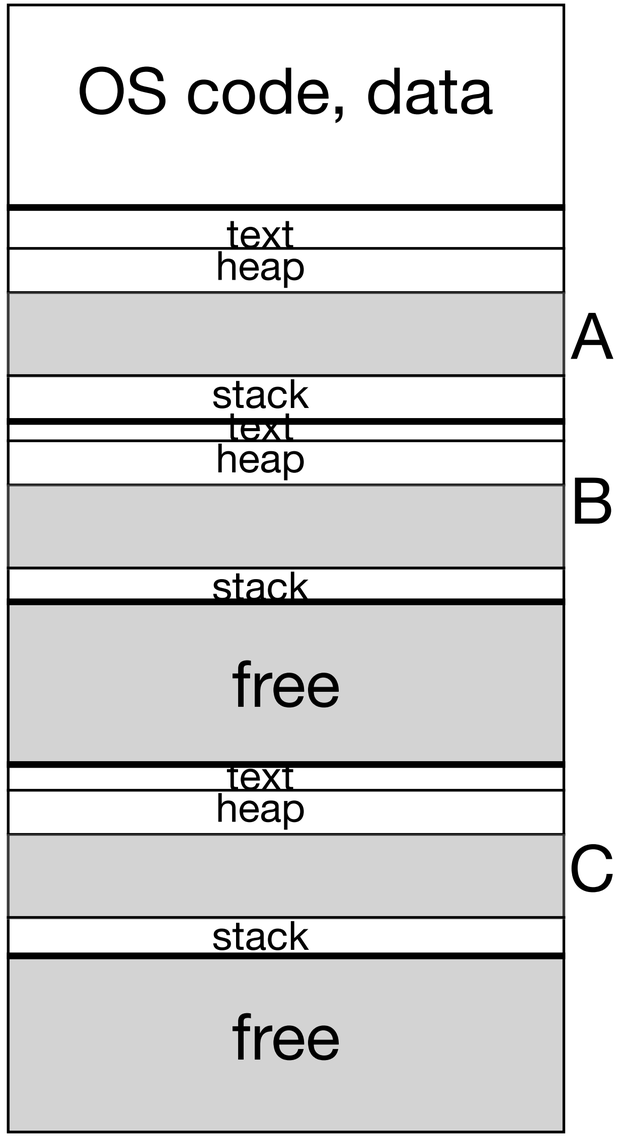
Segmentation

- So far we implicitly assumed that all processes get the same amount of virtual memory
- But within each block there is free space between heap and stack
- The amount of wasted space can be significant
  - e.g. 32-bit address space is 4GB, most programs do not use nearly that much



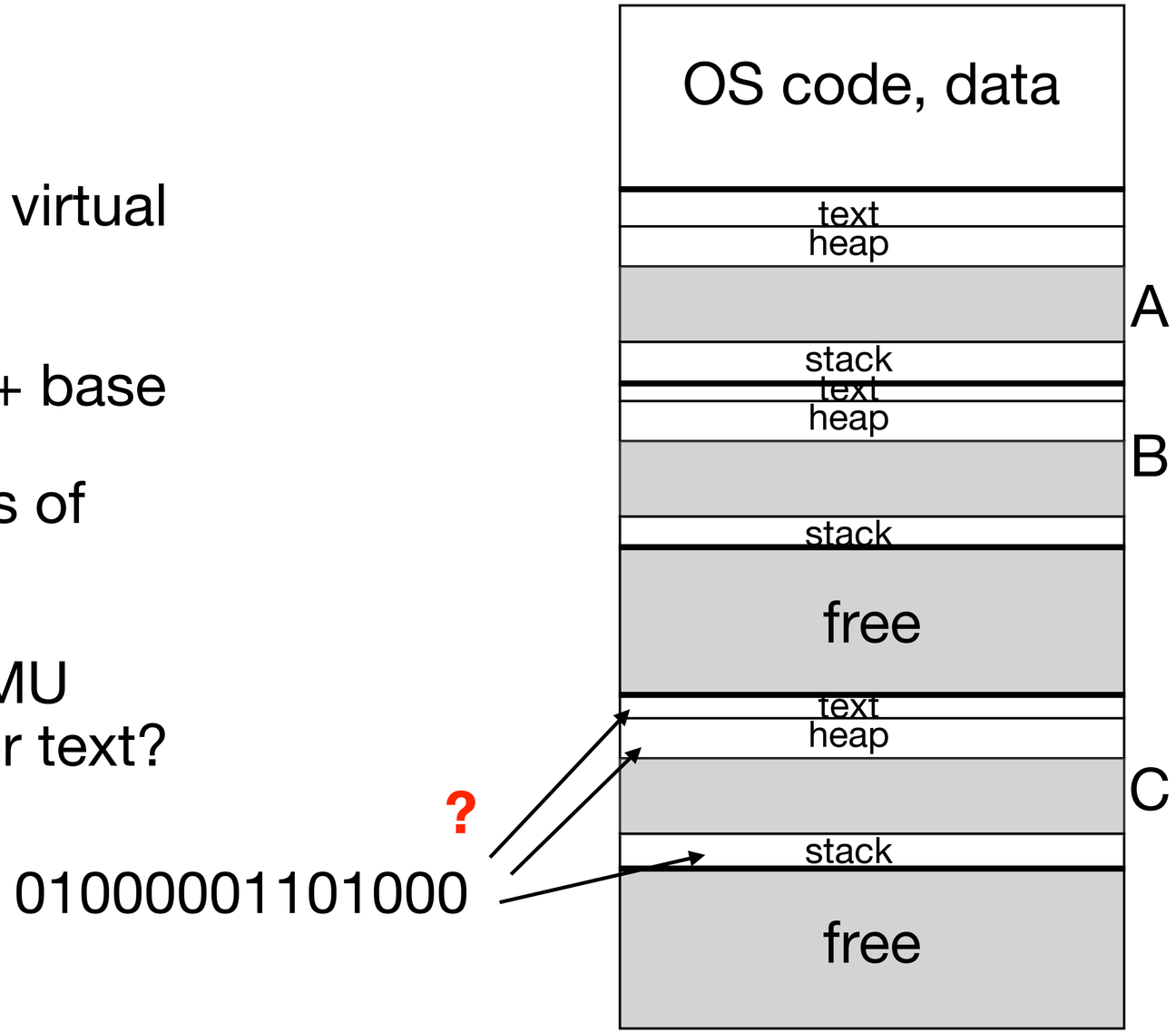
Segmentation

- **Problem #3:** How to allocate space more efficiently to avoid **wasting free space** between heap and stack?
- **Solution: 3 pairs** of base & bound registers in MMU
  - for text, stack, and heap



Segmentation

- Before MMU was just adding base to virtual address
  - physical address = virtual address + base
- Now there are 3 segments and 3 pairs of base & bounds registers
- Given a virtual address, how does MMU know whether it is from stack, heap or text?



Segmentation

- **Problem #4**: how to represent the **type of segment**?
- **Solution:** make it part of the address itself
  - use **two top bits** of the address to represent segment type

Segmentation

- **Problem #4**: how to represent the **type of segment**?
- **Solution:** store it in the address itself
  - use **two top bits** of the address to represent segment type
    - 0 0 for code, 0 1 for heap, 1 1 for stack

virtual address

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0												

segment offset

Segmentation

virtual address

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	1	0	1	0	0	0

segment offset

```
segment = (virtual_address & SEG_MASK) >> SEG_SHIFT
offset = virtual_address & OFFSET_MASK
if (offset > bounds[segment])
    RaiseException(PROTECTION_FAULT)
else
    phys_addr = base[segment] + offset
    register = AccessMemory(phys_addr)
```

segmentation fault*

*initially referred to illegal memory access on a segmented machine, but now applies to most illegal memory operations, even in systems without segmentation

Segfaults

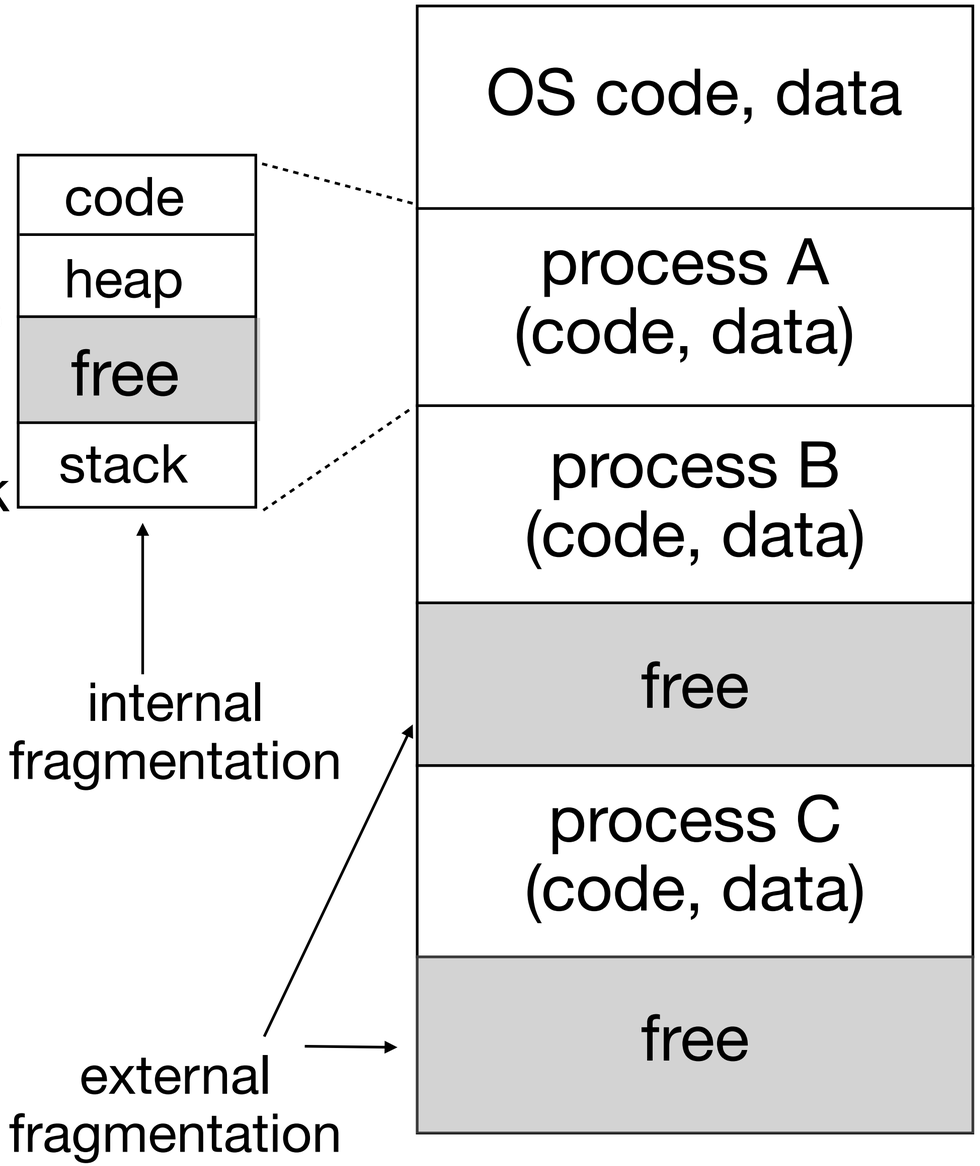
- **Segmentation fault** - accessing memory outside the segment — the term now applies broadly to any memory faults (even in systems with no segmentation)
- Elegant ways of creating segfaults
  - dereferencing a NULL pointer
  - write into the text section
  - accessing unitialized pointer
  - stack overflow (e.g. infinite recursion)
  - stack buffer overflow
  - heap buffer overflow

Summary so far

- **Address translation** is needed to convert virtual addresses of the processes to physical addresses in RAM
  - physical address = virtual address + base
  - bounds for checking the upper limit of memory
  - Physical location of the process memory is recorded in PCB
- **Segmentation** - split virtual memory as 3 chunks - stack, heap and text
  - base and bounds pair for each
  - prevents waste of free space between stack and heap

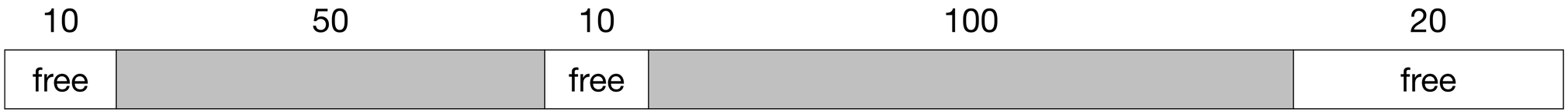
Fragmentation

- **External fragmentation** - memory becomes fragmented after repeated allocations/deallocations of variable size units
  - There is free space, but not in a contiguous block
- **Internal fragmentation** - fragmentation within the allocated block
  - e.g. program requested more than it needs, but the whole space is marked as used
  - segmentation helps with internal fragmentation



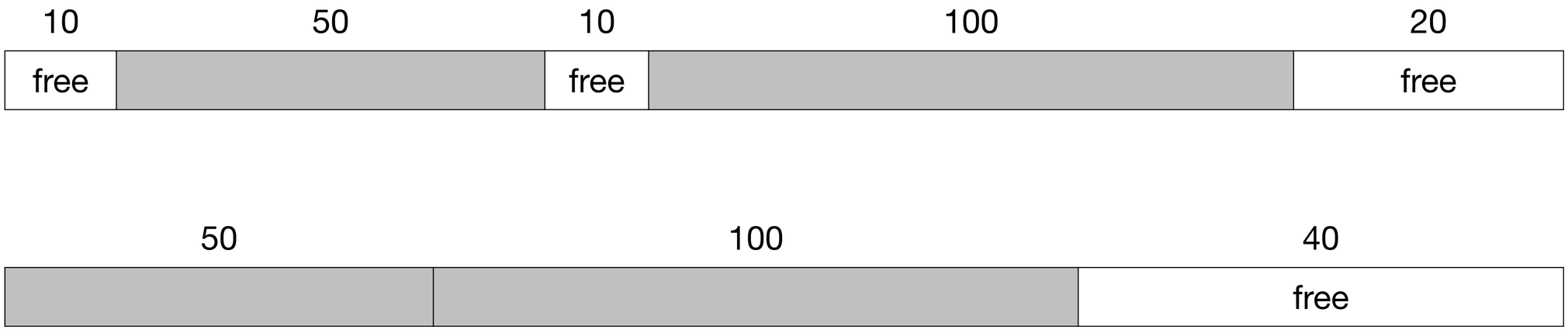
Fragmentation

- **Fragmentation** - memory becomes fragmented after repeated allocations/ deallocations of variable size units
  - There is free space, but not in a contiguous block
  - e.g. OS needs 40 KB of space for a new process
    - there is 40 KB available but divided into 10KB, 10KB and 20KB chunks



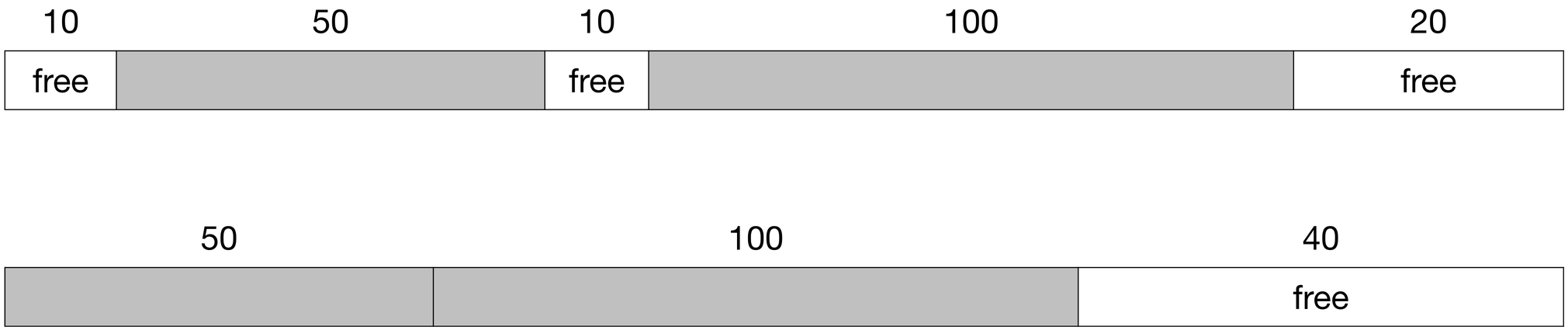
Fragmentation

- **Solution #1: compaction** - rearrange segments to free space
  - need to stop all processes to rearrange memory
  - the operation is expensive



Fragmentation

- **Solution #1: compaction** - rearrange segments to free space
  - need to stop all processes to rearrange memory
  - the operation is expensive
- **Solution #2:** minimize fragmentation by managing free space better



Free-space management

- **Problem:** there is enough total free memory, but no single block enough for a process
- **Solution:** manage free space better to reduce fragmentation
  - **free list** for keeping track of free blocks
  - **split/coalesce blocks** when allocating/deallocating memory
  - **strategy** for choosing the right free block from available ones

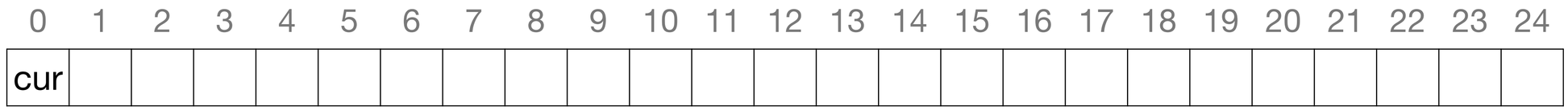
Free-list

think of memory as a flat array with byte-sized cells

for simplicity we only have 25 bytes of memory

heap_start

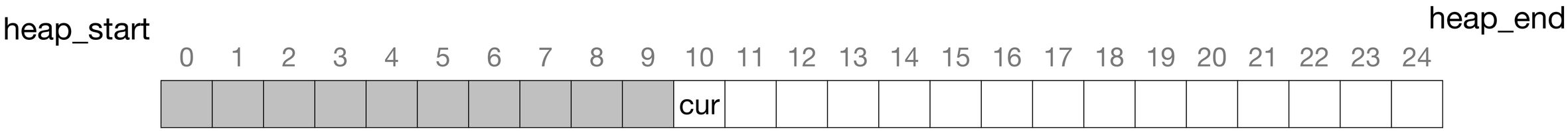
heap_end



cur keeps track of the next address

heap_start and heap_end are pointers to the start and end of the heap

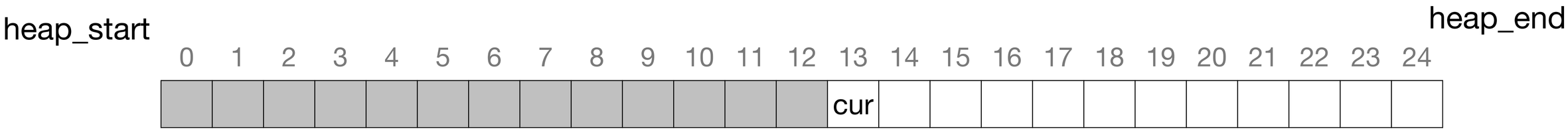
Free-list



ALLOC 10

move the pointer after every allocation

Free-list

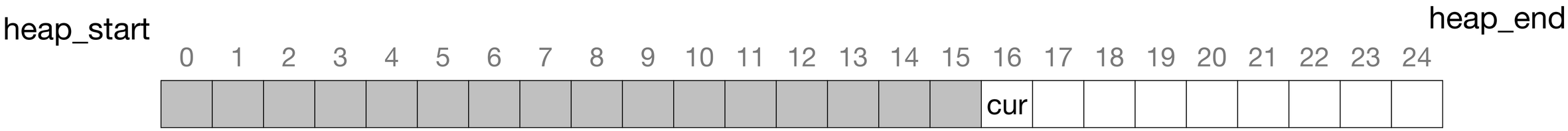


ALLOC 10

ALLOC 3

move the pointer after every allocation

Free-list



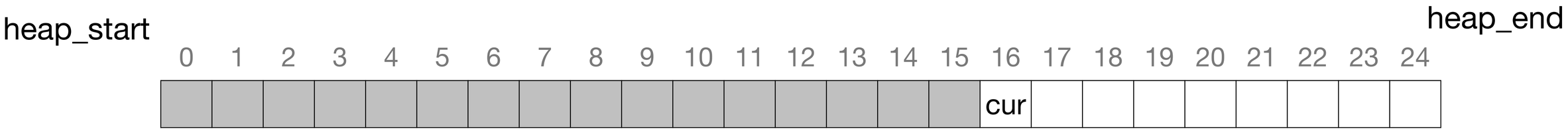
ALLOC 10

ALLOC 3

ALLOC 3

move the pointer after every allocation

Free-list



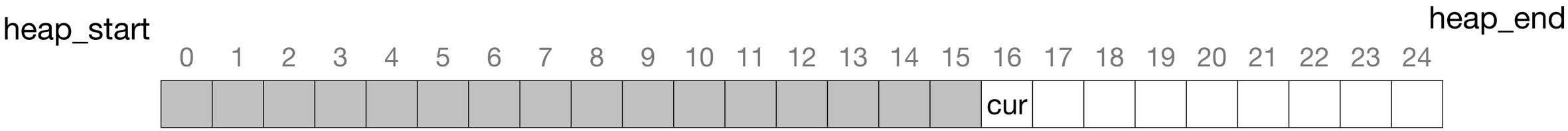
move the pointer after every allocation

ALLOC 10
ALLOC 3
ALLOC 3

```
uint8_t *cur_heap = 0;
void *alloc(size_t size) {
    uint8_t *addr_to_return = cur_heap;
    cur_heap += size;
    return (void*) addr_to_return;
}
```

we use uint8_t but in real world address spaces uint32_t or uint64_t might be needed

Free-list



ALLOC 10

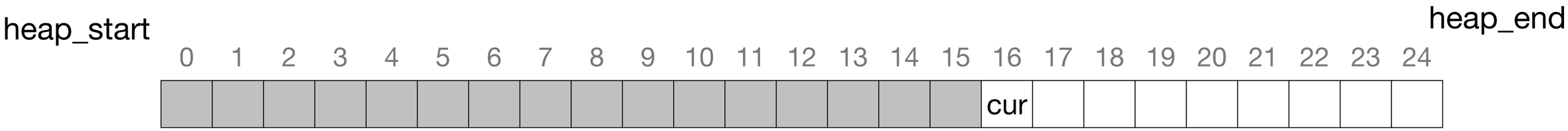
ALLOC 3

ALLOC 3

cur - variable to keep track of the next address

this is called a **bump allocator** - each allocation bumps the cur pointer forward

Free-list



ALLOC 10

ALLOC 3

ALLOC 3

FREE ?

cur - variable to keep track of the next address

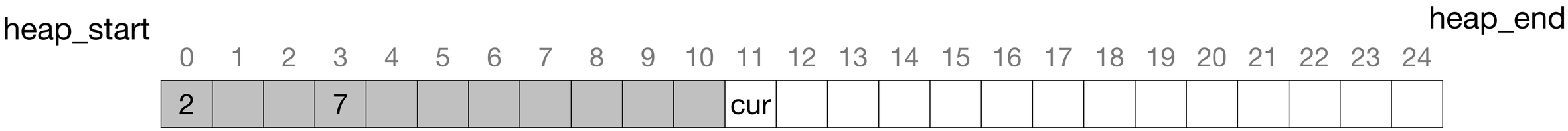
this is called a **bump allocator** - each allocation bumps the cur pointer forward

there is **no free!** we didn't keep track of size

pros: complexity is O(1), no fragmentation

cons: will run out of memory

Free-list



ALLOC 2

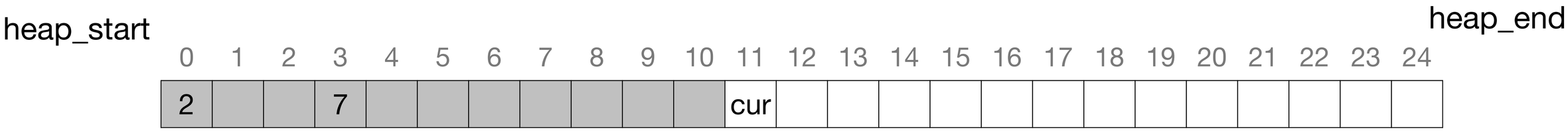
ALLOC 7

FREE ?

record the size of the allocated block at the beginning of each block

need to allocate 1 byte extra for the size

Free-list



ALLOC 2

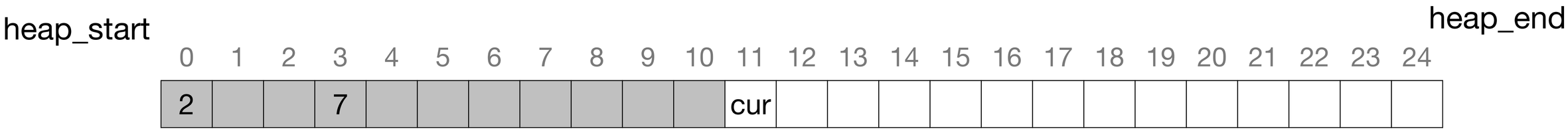
ALLOC 7

FREE ?

```
uint8_t *heap_start = 0;
uint8_t *cur_heap = heap_start;
void *alloc(size_t size) {
    *cur_heap = size;
    cur_heap = cur_heap + 1;
    uint8_t *addr_to_return = cur_heap;
    cur_heap += size;
    return (void*) addr_to_return;
}
```

change the previous function to add 1 to cur pointer to accommodate for stored information

Free-list



ALLOC 2

ALLOC 7

FREE ?

```
uint8_t *heap_start = 0;
uint8_t *cur_heap = heap_start;
void *alloc(size_t size) {
    *cur_heap = size;
    cur_heap = cur_heap + 1;
    uint8_t *addr_to_return = cur_heap;
    cur_heap += size;
    return (void*) addr_to_return;
}
```

can we free now?
still no, because we cannot tell which block is used/free

Free-list

note that now for every allocated block there are 2 bytes
holding the information about its size and status

heap_start

heap_end

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
2	U			7	U								cur											

ALLOC 2

ALLOC 7

FREE ?

record the size of the allocated block at the beginning of each block

record whether the block is used or not

U - used, F - free

need to allocate 2 bytes extra for size and status

Free-list

note that now for every allocated block there are 2 bytes holding the information about its size and status

heap_start

heap_end

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
2	U			7	U								cur											

ALLOC 2

ALLOC 7

FREE ?

```
#define USED 1
#define FREE 0

uint8_t *heap_start = 0;
uint8_t *cur_heap = heap_start;
void *alloc(size_t size) {
    *cur_heap = size;
    cur_heap += 1;
    *cur_heap = USED;
    cur_heap += 1;
    uint8_t *addr_to_return = cur_heap;
    cur_heap += size;
    return (void*) addr_to_return;
}
```

can we free now?

change the previous function to add 2 to cur pointer to accommodate for size and status

Free-list

note that now for every allocated block there are 2 bytes holding the information about its size and status

heap_start

heap_end

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
2	U			7	U								cur											

ALLOC 2

ALLOC 7

FREE ?

```
#define USED 1
#define FREE 0

uint8_t *heap_start = 0;
uint8_t *cur_heap = heap_start;
void *alloc(size_t size) {
    *cur_heap = size;
    cur_heap += 1;
    *cur_heap = USED;
    cur_heap += 1;
    uint8_t *addr_to_return = cur_heap;
    cur_heap += size;
    return (void*) addr_to_return;
}
```

can we free now?

YES, just need to update the block status

```
void free(void* ptr) {
    if (*(ptr -1) == USED)
    {
      *(ptr - 1) = FREE;
    }
}
```

Free-list

heap_start

heap_end

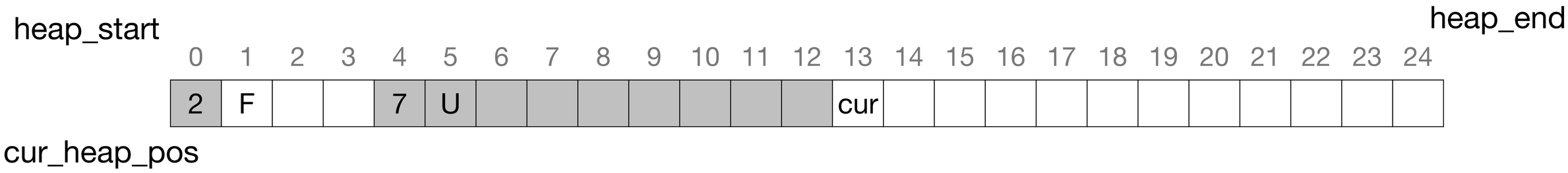
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
2	F			7	U								cur											

y= ALLOC 2
ALLOC 7
FREE y

we can free now!
ALLOC 2 will return a pointer to address y
check the address at y-1
if y-1 is marked as U (used), set it to F
do not update the size

```
void free(void* ptr) {
    if (*(ptr -1) == USED)
    {
      *(ptr - 1) = FREE;
    }
}
```

Free-list



how can we reuse the freed space?

ALLOC 2

ALLOC 7

FREE 2

ALLOC 1

need to traverse the list and find free region large enough

Free-list

heap_start

heap_end

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
2	F			7	U								cur											

cur_heap_pos

```
#define USED 1
#define FREE 0
uint8_t *heap_start = 0;
uint8_t *cur_heap = heap_start;
void *alloc(size_t size) {
    *cur_p = heap_start;
    while (cur_p < cur_heap) {
      cur_size = *cur_pointer;
      status = *(cur_pointer + 1);

      if cur_size >= size && status == FREE) {
        status = USED;
        return cur_p + 2;
      }
      cur_p = cur_p + (size + 2);
    …
}
```

traverse the heap by moving the current pointer until free block is found

ALLOC 2

ALLOC 7

FREE 2

ALLOC 1

the rest of the previous alloc function goes here unchanged

Free-list

- The allocator we just wrote is a naïve version of `malloc()` and `free()`
- **Free list** is used for
  - **allocating** new memory
  - **freeing** allocated memory
  - **reusing** freed memory (heap traversal)

Free-list

- Free-list is **embedded** in the heap for efficiency
  - size and status (free or used) are stored before each allocated block
- **Pros:** time and space **efficient**
- **Cons:** need extra checks to maintain heap **integrity** against
  - heap buffer overflows — can corrupt block information
  - freeing the wrong pointer — free will check neighboring addresses which may contain incorrect information
  - double-freeing - free might access something it shouldn’t

Fragmentation

- Free-list **does not** eliminate fragmentation
- Two mechanisms can help minimize it
  - **splitting** and **coalescing**
  - allocation **strategy**

Coalescing

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
6	F							6	F							6	F							

a ALLOC 6
b ALLOC 6
c ALLOC 6
FREE c
FREE b
FREE a
a ALLOC 7

say we perform operations in this order

the last operation will fail -> there is no block large enough

Coalescing

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
6	U							6	U							6	U							

a ALLOC 6

b ALLOC 6

c ALLOC 6

FREE c

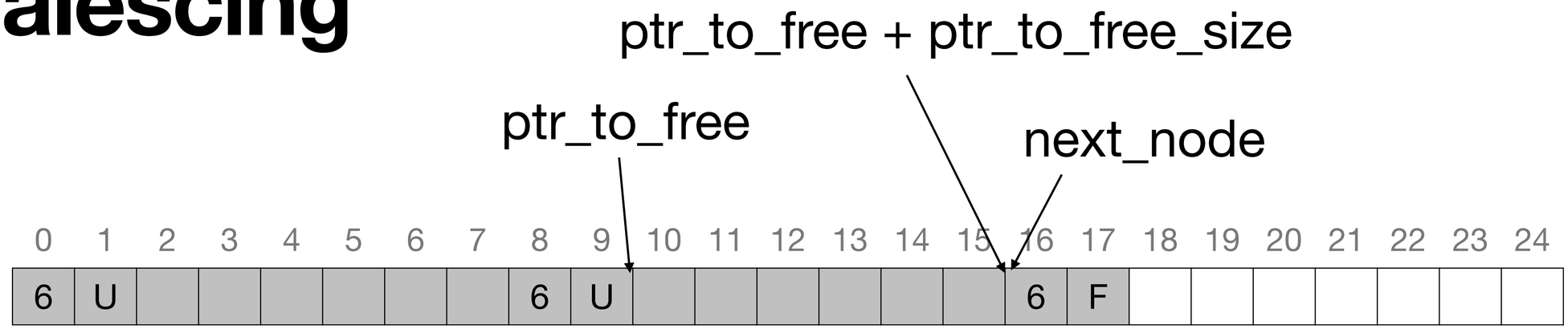
FREE b

FREE a

a ALLOC 7

coalescing - merging neighbouring free blocks

Coalescing



a ALLOC 6
b ALLOC 6
c ALLOC 6
FREE c
FREE b
FREE a
a ALLOC 7

coalescing - merging neighbouring free blocks

when calling free():
if ptr_to_free + ptr_to_free_size == next_node
merge nodes into one and update the size

coalescing works only on neighbouring blocks - may still leave small blocks of free memory

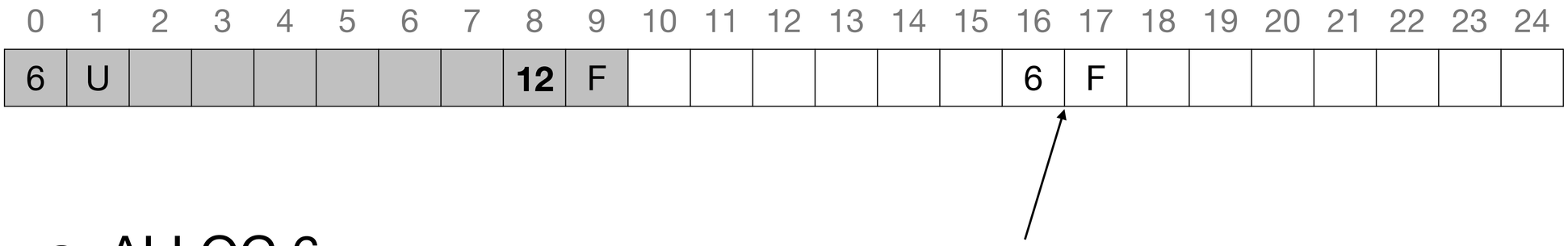
Coalescing

coalescing - merging neighbouring free blocks

a ALLOC 6
b ALLOC 6
c ALLOC 6
FREE c
FREE b
FREE a
a ALLOC 7

when calling free():
look at the next node
if next node is free
if ptr_to_free + ptr_to_free_size == next_node
look at the previous node
if prev_node + prev_node_size + sizeof(heap_node) == ptr_to_free
merge nodes by updating the size of the node and mark the new node free

Coalescing

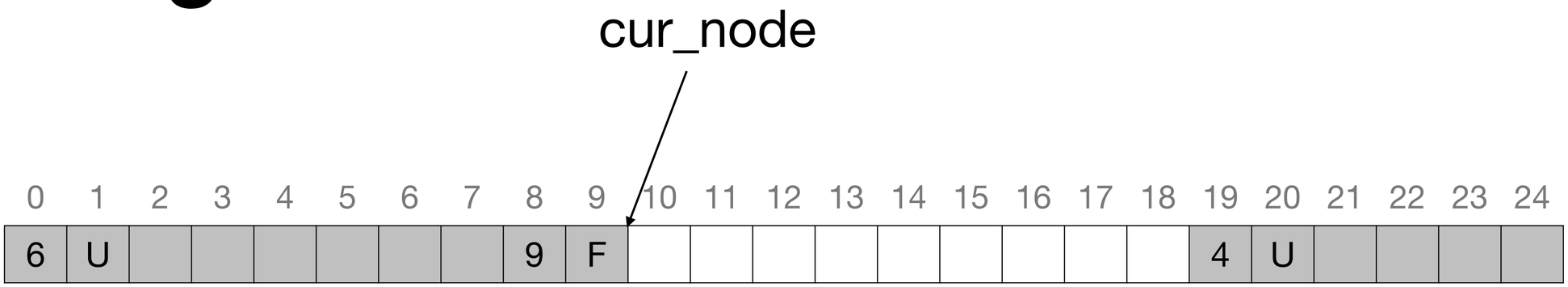


a ALLOC 6
b ALLOC 6
c ALLOC 6
FREE c
FREE b
FREE a
a ALLOC 7

free does not erase anything from freed chunk of memory, just marks the block as free

e.g. block information will stay after free is done and can be overwritten if this space is used again

Splitting



a ALLOC 3

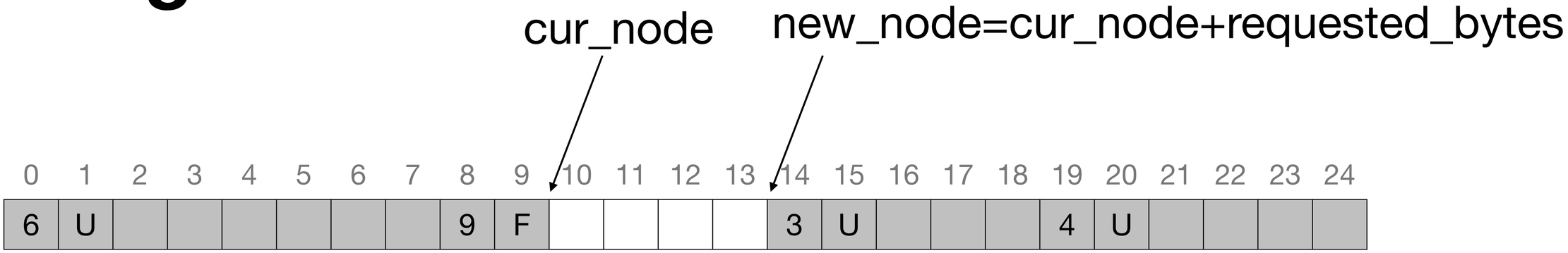
splitting - when allocating space smaller than the free block

find node big enough to hold the request (cur_node)

create a new node at cur_node + requested_bytes

update this node's size

Splitting



a ALLOC 3

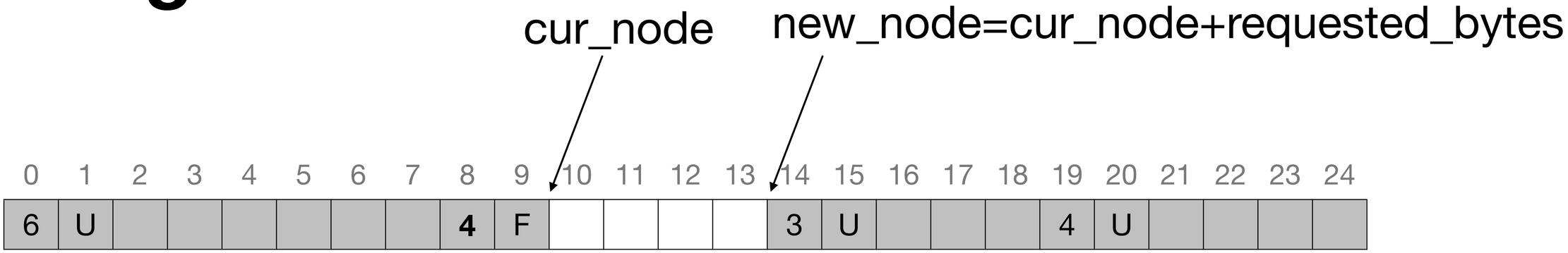
splitting - when allocating space smaller than the free block

find node big enough to hold the request (cur_node)

create a new node at cur_node + requested_bytes

update this node's size

Splitting



a ALLOC 3

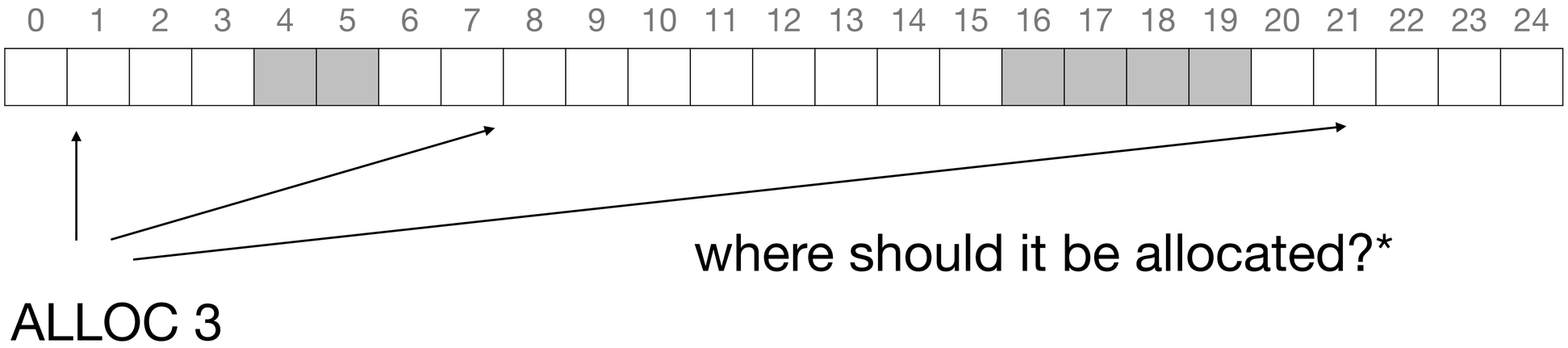
splitting - when allocating space smaller than the free block

find node big enough to hold the request (cur_node)
create a new node at cur_node + requested_bytes
update current node's size

Fragmentation

- Free-list **does not** eliminate fragmentation
- Two mechanisms can help minimize it
  - **splitting** and **coalescing** - splitting and merging **neighboring** free blocks
  - allocation **strategy** - how to pick a next block?

Allocation strategies



*normally, when making allocations, you will also take into account additional bytes for storing the status bits. So if we need to allocate 3 units of memory, we need to look for the free space that is 3 units + 2 bits to fit it

In this example, we are ignoring this for simplicity

Allocation strategies

where should it be allocated?

ALLOC 3

Option #1 Best fit - the smallest block that can fit the request

pros: less wasted space

cons: need to traverse the whole heap

Allocation strategies

where should it be allocated?

ALLOC 3

Option #2 Worst fit - the largest block that can fit the request
pros: leaves larger chunks for future allocations
cons: need to traverse the whole heap

Allocation strategies

where should it be allocated?

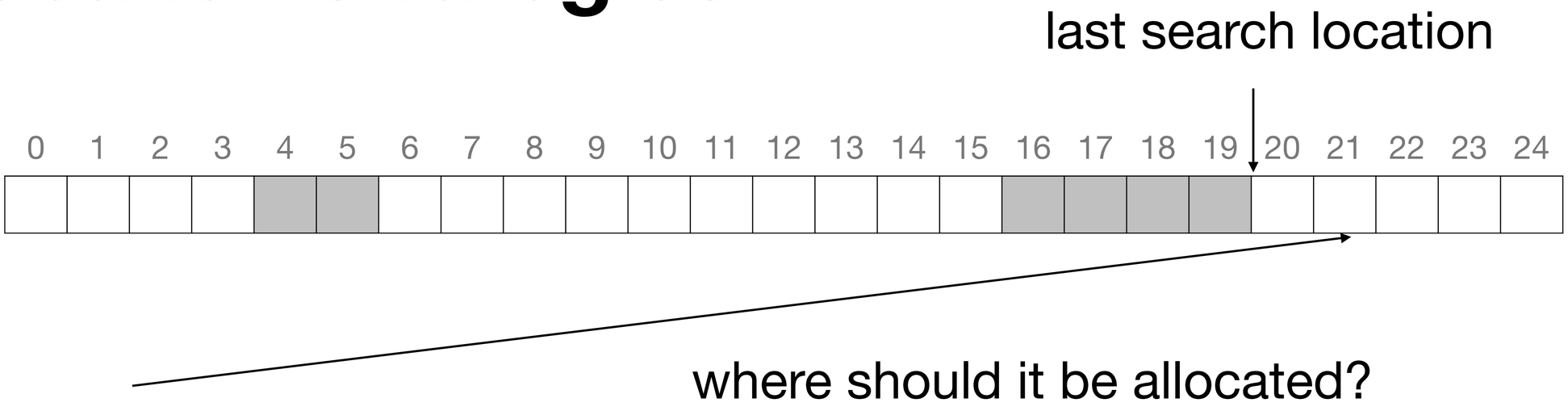
ALLOC 3

Option #3 First fit - the first block can fit the request

pros: fast, do not need to traverse heap

cons: many small gaps in the beginning of the freelist

Allocation strategies



ALLOC 3

Option #4 Next fit - same as first-fit but continue from the last location

pros: spreads search to the entire list, but no exhaustive traversal needed

cons: may skip nodes allocated before the last search location

Allocation strategies

- **Best fit** - return block closest in size
- **Worst fit** - return the largest block
- **First fit** - return the first large enough block
- **Next fit** - like first fit, but start searching where it was last left off
- More sophisticated strategies
  - **segregated lists** - maintain several lists for requests of different sizes
  - **binary buddy allocation** - instead of traversing, recursively divide memory by two until a block large enough is found

Summary

- **Address translation** - translating virtual addresses of the processes to physical memory addresses
  - **base and bounds** - hardware registers that hold min and max location of the process' virtual memory
  - **segmentation** - splitting virtual memory into segments (heap, stack, text)
- **Fragmentation** - "holes" in memory from allocations/deallocations
  - Internal - holes in the allocated memory
  - External - holes in the physical memory
- **Free-list** - a data structure for keeping track of free space, embedded in memory
- **Allocation strategies** - how to pick free space for allocation (first, next, best, worst)

Reading

- Chapter 15 Mechanism: Address Translation
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-mechanism.pdf
- Chapter 16 Segmentation
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-segmentation.pdf
- Chapter 17 Free-space management
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-freespace.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-segmentation
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-freespace
- No optional reading this week

A1

- Reminder: Use of LLMs is **not allowed** for any part of the assignments
- Do not leave A1 until last moment
- I will not be troubleshooting autograder/A1 issues over the weekend

Announcements

- **SOCIS URA/USRA Mixer**
  - **today**, 1-3pm @ REYN1101
  - No office hours on today
- Notetakers needed
  - Please sign up if you are taking notes already

Quiz 6

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999